LSTM: A Search Space Odyssey

Paper Summary and From-Scratch Implementation

Jibran Lakhdhir

August 7, 2026

Abstract

Greff et al.'s "LSTM: A Search Space Odyssey" (2015) is a large-scale empirical ablation study of the Long Short-Term Memory (LSTM) recurrent neural network architecture. Rather than proposing a new architecture, the paper audits the components accumulated by the "vanilla" LSTM since its original 1995/1997 formulation — the forget gate, peephole connections, and full backpropagation through time — via 5,400 controlled experiments (approximately 15 CPU-years) across three sequence-modeling tasks (speech, handwriting, and music). The central finding is that none of eight tested architectural variants significantly improves upon the standard LSTM, and that the forget gate and output activation function are its two most critical components. This report summarizes the paper's core architecture and findings, and describes a from-scratch PyTorch implementation of the vanilla LSTM cell alongside two of the paper's simplification variants — CIFG (Coupled Input–Forget Gate) and NP (No Peepholes) — trained on a synthetic long-range copy task designed to directly stress-test the mechanism the paper identifies as most important: the cell state's resistance to information loss over time.

1 Problem and Motivation

Recurrent neural networks (RNNs) process sequential data by carrying a hidden state forward from one time step to the next, allowing earlier context to influence later predictions. Simple RNNs (SRNs), however, struggle to preserve information across long time gaps: because the hidden state is repeatedly passed through a weight matrix and a saturating nonlinearity at every step, gradients computed via backpropagation through time (BPTT) tend to vanish (or occasionally explode) as the number of steps grows, making it very difficult to learn dependencies that span more than a short window.

The LSTM architecture (Hochreiter & Schmidhuber, 1997) addresses this by introducing a dedicated *cell state* — a separate memory pathway alongside the hidden state, protected by learned gating units that control what information is written, erased, and read out at each time step. Between 1997 and roughly 2005, several modifications were proposed independently by different research groups (the forget gate, peephole connections, and a shift to exact full-gradient BPTT training), and by the time of this paper's publication, "LSTM" had come to mean somewhat different things across the literature. Greff et al. define *vanilla LSTM* as the specific combination of these additions that had become the de facto standard, and then systematically ask which of its components actually earn their place — a question that had not previously been answered with controlled, large-scale evidence.

2 Architecture

2.1 The Cell State and Gating Mechanism

The core structural idea is a memory cell c^t that can maintain its value over time, regulated by three gates: an **input gate** controlling how much new information is written, a **forget gate** controlling how much old information is retained, and an **output gate** controlling how much of the cell's contents are exposed as the block's output at the current time step. A separate **block input** computes the candidate content proposed for writing at each step, and optional **peephole connections** allow the gates to observe the cell's own value directly, rather than only the current input and previous output.

2.2 Forward Pass Equations

For a layer of N LSTM blocks receiving M-dimensional inputs x^t, with previous block output $y^{t-1} \in \mathbb{R}^N$ and previous cell state $c^{t-1} \in \mathbb{R}^N$:

$$\bar{z}^t = W_z x^t + R_z y^{t-1} + b_z, \qquad z^t = g(\bar{z}^t) \quad \text{(block input)} \tag{1}$$

$$\bar{i}^t = W_i x^t + R_i y^{t-1} + p_i \odot c^{t-1} + b_i, \qquad i^t = \sigma(\bar{i}^t) \quad \text{(input gate)} \tag{2}$$

$$\bar{f}^t = W_f x^t + R_f y^{t-1} + p_f \odot c^{t-1} + b_f, \qquad f^t = \sigma(\bar{f}^t) \quad \text{(forget gate)} \tag{3}$$

$$c^t = z^t \odot i^t + c^{t-1} \odot f^t \qquad \text{(cell state)} \tag{4}$$

$$\bar{o}^t = W_o x^t + R_o y^{t-1} + p_o \odot c^t + b_o, \qquad o^t = \sigma(\bar{o}^t) \quad \text{(output gate)} \tag{5}$$

$$y^t = h(c^t) \odot o^t \qquad \text{(block output)} \tag{6}$$

where σ is the logistic sigmoid (used for gates, since their output represents a proportion in $(0, 1)$) and g, h are typically tanh (used for content-carrying values, which may be signed). The cell update (Equation 4) is the paper's "Constant Error Carousel" (CEC): because c^{t-1} enters via an elementwise, additively-gated term rather than a matrix multiply followed by a nonlinearity, gradients can flow backward through many time steps without the multiplicative decay that afflicts SRNs, provided the forget gate remains close to 1.

2.3 Ablation Variants

The paper defines eight single-component modifications of vanilla LSTM. This implementation focuses on two, chosen because the paper's own results found both to be safe simplifications that reduce parameter count without significantly harming performance:

- **CIFG (Coupled Input–Forget Gate):** the forget gate is not learned independently; instead $f^t = 1 - i^t$. This ties writing and erasing into a single decision and is mathematically equivalent to the gating mechanism used in GRUs.
- **NP (No Peepholes):** the peephole terms (p_i, p_f, p_o) are removed entirely; gates are computed only from the current input and previous output, identically to the block input.

3 Implementation Details and Deviations from the Paper

The vanilla LSTM cell and both ablation variants were implemented from scratch in PyTorch, using explicit tensor operations for every gate computation rather than `nn.LSTM`, so that each equation above corresponds directly to a line of code.

Task. The paper evaluates on TIMIT (speech), IAM Online (handwriting), and JSB Chorales (music) — all substantial, domain-specific datasets requiring dataset-specific preprocessing pipelines. For this implementation, all three architectures were instead evaluated on a synthetic **long-range copy task**: the model is given a short sequence of random tokens, followed by a delay period of blank filler tokens, followed by a delimiter, and must reproduce the original tokens immediately after the delimiter. This task was chosen deliberately because it isolates exactly the capability the paper identifies as most dependent on the forget gate and output activation: preserving information across a temporal gap with no useful signal in between.

Scale. Hidden size, batch size, and training step count were all substantially reduced relative to the paper's setup (which used random search over 200 trials per variant per dataset, for a total of 5,400 experiments). This implementation trains each variant once, with fixed hyperparameters (hidden size = 32, learning rate = 10^{-3}, batch size = 64, 300 training steps), rather than performing a hyperparameter search.

Controlled ablation. Following the paper's own experimental logic, Vanilla, CIFG, and NP were trained using an identical training procedure, identical hyperparameters, and an identical random seed (governing both weight initialization and the sequence of training batches), so that any difference in outcome can be attributed to the single architectural change distinguishing each variant, rather than to confounding differences in setup.

4 Results

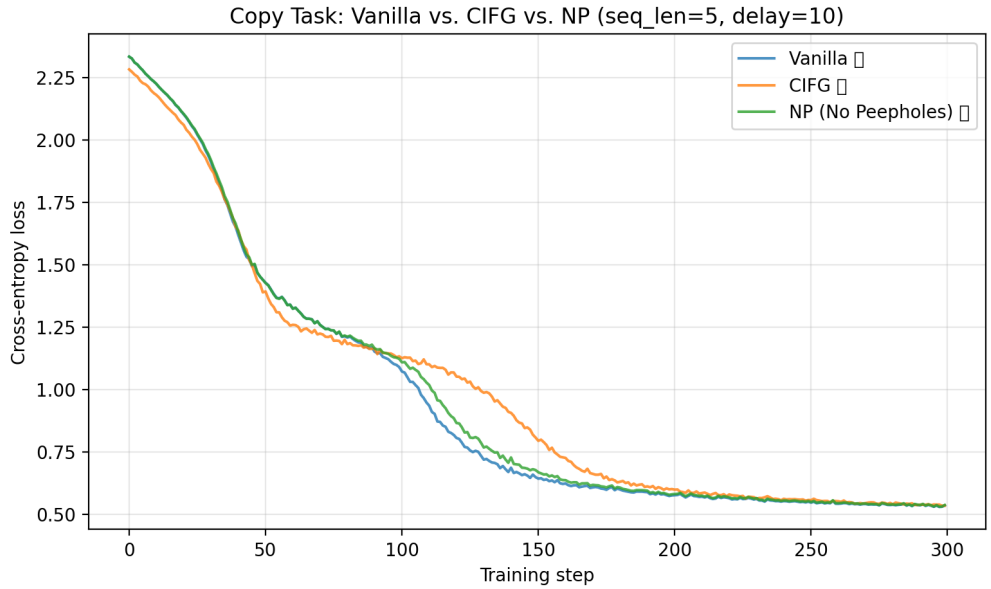


Figure 1: Training loss for Vanilla, CIFG, and NP LSTM cells on the copy task (content length = 5, delay = 10).

After 300 training steps, final training loss was nearly identical across all three variants: Vanilla achieved 0.5351, CIFG achieved 0.5344, and NP achieved 0.5369 — a spread of less than 0.003 between the best and worst performing variant. This is directly consistent with the paper's own finding (Section V-A) that coupling the input and forget gates (CIFG) and removing peephole connections (NP) do not significantly harm performance relative to vanilla LSTM. At small scale,

on a synthetic task, both simplifications reproduce the paper's headline result: peepholes and an independently-learned forget gate are not load-bearing components, and removing either did not measurably hurt the model's ability to solve the copy task within this training budget.

It is worth noting that a loss of approximately 0.53 indicates the task was *not fully solved* within 300 steps — all three variants still had room to improve, rather than having converged to near-zero loss. Given the near-identical final losses across variants, however, this under-training affected all three equally and does not undermine the ablation comparison itself.

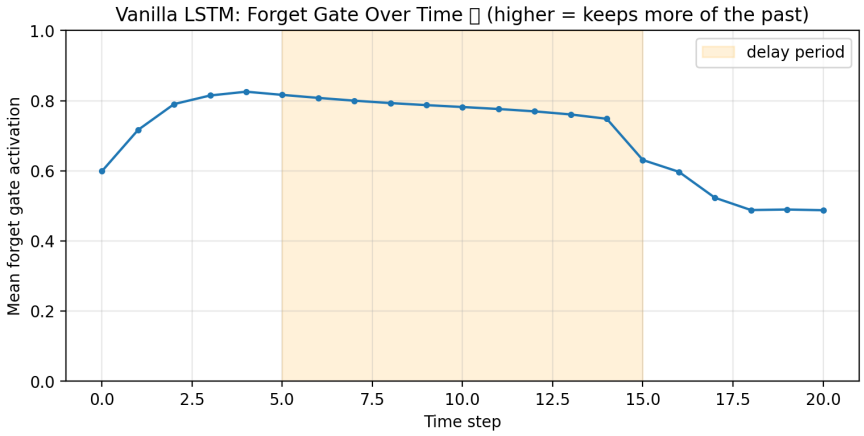


Figure 2: Mean forget gate activation over time for the Vanilla LSTM on a single example sequence. The shaded region indicates the delay period.

Figure 2 shows the mean forget gate activation over time for a single example sequence processed by the trained Vanilla LSTM. The forget gate rises quickly from approximately 0.60 at $t = 0$ to approximately 0.80–0.83 by $t = 3$–4, coinciding with the presentation of the content tokens to be copied. During the delay period ($t = 5$ through $t = 14$, shaded), the forget gate remains elevated, staying in the range of roughly 0.75–0.82 throughout — consistent with the Constant Error Carousel mechanism described in Section II: a forget gate value well above 0.5 during the delay indicates the network is preferentially retaining the cell state's contents rather than overwriting them, which is precisely the behavior required to solve the copy task across a 10-step gap with no useful input signal.

Two aspects of the plot are worth noting as deviations from an idealized picture. First, the forget gate does not approach 1.0 during the delay, as a theoretically "perfect" memory-preservation strategy might suggest; a value around 0.8 still permits some decay of the retained signal over the 10-step delay, which is plausibly connected to the model's relatively high final loss (0.5351) after only 300 training steps — a more fully converged model might be expected to push this value closer to 1. Second, the forget gate drops sharply once the delay period ends and the copying phase begins ($t = 15$ onward), falling to approximately 0.5–0.6. This is a sensible learned behavior rather than a flaw: once the content tokens have been successfully read out, there is no longer any reason for the network to continue protecting the now-unneeded stored values, and the gate is free to begin admitting new information for whatever follows.

Variant	Final Training Loss	Notes
Vanilla	0.5351	Baseline
CIFG	0.5344	Forget gate coupled to input gate ($f^t = 1 - i^t$)
NP	0.5369	Peepholes removed

Table 1: Summary of final training loss across variants after 300 steps. Differences between variants are small relative to each variant's absolute loss, consistent with the paper's finding that these simplifications do not significantly harm performance.

5 Code

Full implementation available at: LSTM Colab Notebook.